Lecture 06: POS tagging

Natural Language Processing | 30 October 2026

> These study notes are synthesized from the 62-page lecture slide deck. No recording was available, so the explanations describe the slides rather than reproduce a spoken lecture.

Lecture overview

Part-of-speech tagging assigns a grammatical label to each word in context. The lecture builds from a generative hidden Markov model (HMM), through Viterbi decoding and conditional random fields (CRFs), to bidirectional LSTMs and Transformer-based taggers. The central idea is that a useful tagger must combine word-level evidence with the constraints and context of the full sequence.

Key ideas

1. A word can have different tags in different sentences, so context matters.
2. HMM probabilities and Viterbi decoding make globally coherent tag sequences practical to find.
3. CRFs and neural encoders use richer evidence while modeling relationships among neighboring labels.

Slide-by-slide explanations

01. What POS tagging does

Part-of-speech (POS) tagging assigns each token a grammatical category such as noun, verb, determiner, or adjective. The task is sequence labeling: infer one tag for every word using lexical evidence and surrounding context.

02. Lecture roadmap

The lecture moves from the tagging problem and tagsets to probabilistic sequence models (HMMs), dynamic programming with Viterbi, conditional random fields (CRFs), and neural models. The progression is from hand-defined probabilities to learned contextual representations.

03. Words in, tags out

Given words x1 through xn, predict tags y1 through yn; the output aligns one label with each input token. Because a decision depends on both the word and its context, the model should choose a coherent tag sequence.

04. Open and closed word classes

Open-class words (nouns, verbs, adjectives, adverbs, numerals, and interjections) readily gain new members. Closed-class function words (pronouns, determiners, auxiliaries, conjunctions, prepositions, and particles) form smaller, more stable sets. These classes offer useful clues but do not resolve every ambiguous word.

05. Words can have multiple tags

The same spelling can take different tags: book is a noun in “I read a book” and a verb in “I will book a flight”. A tagger cannot assign a fixed label to a word type; it must use sentence context.

06. Ambiguous words are frequent

Corpus counts reveal an imbalance: most distinct word types occur with only one tag, yet common ambiguous types appear so often that they account for many tokens. A system must handle ambiguity even if it affects a minority of vocabulary entries.

07. A strong baseline, with limits

A baseline labels each word with its most frequent tag in the training data. The cited result reaches about 92% token accuracy on WSJ, but only 55% of sentences are entirely correct; performance also drops in new domains and languages.

08. Tagging supports parsing

POS tags add lexical categories before a parser builds dependencies. In the example, labels help identify “Who” as the subject, “developed” as the verb, and the object and its modifiers. Parsing can then connect words into grammatical relations.

09. Tagging supports translation

French commonly places an adjective after its noun (“la pomme rouge”), while English usually places it before (“the red apple”). POS tags identify nouns and adjectives so a translation system can reorder them appropriately.

10. Tagging supports sentiment analysis

POS tags can support sentiment analysis by locating adjectives in reviews. The example labels evaluative words such as “amazing”, “good”, “bad”, and “cheap”, separating them from nouns, verbs, and function words so downstream analysis can focus on useful cues.

11. Tagging supports speech synthesis

Text-to-speech needs more than spelling: the pronunciation of words such as “lead”, “resume”, and “report” can depend on their grammatical role and context. POS tags help select the intended pronunciation and prosody.

12. Sequence tagging as a general task

POS tagging is one example of sequence tagging: assign a label to each token while preserving token order. The same model ideas can support named entity recognition and other word-level labeling tasks.

13. Named entity recognition example

Named entity recognition marks spans such as dates, organizations, people, and works of art. The annotated sentence shows that labels attach to tokens and may span multiple words, making NER another sequence-labeling problem.

14. BIO labels mark entity spans

BIO encoding marks the first token of an entity with B, later tokens with I, and tokens outside entities with O. In the example, “New York” is a location span and “Michael Jeffrey Jordan” is a person span. Token labels can therefore represent multiword boundaries.

15. Universal Dependencies tagset

Tagsets differ in size and detail. This lecture uses Universal Dependencies, a cross-linguistic set of 17 coarse-grained POS categories. A shared inventory makes comparisons across languages easier, while giving up some fine distinctions.

16. HMM roadmap

This section introduces hidden Markov models as a generative approach to POS tagging, then shows how Viterbi finds the best tag path. Later sections replace the HMM’s limited feature set with CRFs and neural representations.

17. Evidence for a tag

Evidence comes from a word’s identity and neighboring tags. “Arrow” strongly suggests a noun; “book” is ambiguous. Tag transitions also help: determiners usually precede adjectives or nouns, while two determiners rarely occur consecutively.

18. HMM generative process

The HMM generates a sentence by choosing each tag from the preceding tag, then choosing a word from the current tag. This makes tags a first-order Markov chain and treats word emissions as conditionally independent given the tags.

19. Joint probability of words and tags

The joint score for a sentence and tag sequence multiplies each transition $P(t_i|t_{(i-1)})$ by each emission $P(w_i|t_i)$, then includes an end transition. Comparing candidate tag paths uses both grammatical sequence likelihood and word-tag compatibility.

20. Tags as Markov states

View each tag as a state in a Markov chain. Arcs encode transition probabilities between tags; the path begins at a start state and ends at an end state. The chain captures local grammatical patterns such as likely neighboring categories.

21. Transition probabilities

Rows condition on the previous tag and columns list the next tag. Corpus estimates show patterns such as proper nouns often starting sentences, auxiliaries commonly preceding verbs, and adjectives often followed by nouns. Each complete row should form a probability distribution.

22. Transition table completeness

The teaching table intentionally omits many entries, so a visible row does not sum to one. A valid transition model must include all possible next tags, and the probabilities in each row must sum to one.

23. Words emitted from tag states

An HMM state also emits an observed word. P(w_i|t_i) is the emission probability: it measures how likely a word is under a tag, complementing the transition model's probability of moving between tags.

24. Emission probabilities

Emission estimates connect words to tags: "Janet" is likely under PROPN, "will" under AUX, and "the" under DET. Words can still have several possible tags, so the emission distribution is not a deterministic dictionary.

25. Maximum-likelihood estimates

Maximum-likelihood estimates use corpus counts. Divide a tag-transition count by the count of its previous tag; divide a tag-word count by the count of that tag. These normalized frequencies estimate transition and emission probabilities.

26. HMM dependency diagram

In the graphical model, shaded nodes are observed words and unshaded nodes are hidden tags. Arrows show conditional dependencies: each tag depends on the previous tag, and each word depends on its tag. This diagram encodes the HMM assumptions.

27. Scoring a tagged sentence

To score "Unicorns bring joy" with NOUN-VERB-NOUN, multiply the start-to-NOUN transition, the NOUN-to-VERB and VERB-to-NOUN transitions, each word's emission under its proposed tag, and the final transition. A single path receives one joint probability.

28. The hidden tag path

At prediction time the words are visible but their tags are hidden. The goal is to find the most probable state path that could have emitted those words. "Hidden" describes unavailable labels; "Markov" describes limited dependence on previous tags.

29. Viterbi roadmap

This section shifts from scoring a proposed tag sequence to searching for the best sequence. The next slides derive the objective and compare exhaustive search, greedy tagging, and Viterbi dynamic programming.

30. Find the most probable tag sequence

Choose T to maximize P(T|W). By Bayes' rule this is proportional to P(W|T)P(T); P(W) is constant across candidate tag sequences, so it can be dropped for the argmax. The model combines fit to the words with the prior probability of the tag sequence.

31. Transition and emission factors

In the HMM, P(T) factors into tag transitions and P(W|T) factors into emissions. Each path score is therefore a product of local transition and emission factors. This factorization makes efficient dynamic programming possible.

32. Why not enumerate every sequence?

One simple idea is to enumerate every possible tag sequence, score each one, and choose the best. That is correct in principle, but the number of alternatives grows quickly with sentence length, making exhaustive search impractical.

33. The search space grows exponentially

With c tag choices at each of n positions, there are c^n possible sequences. Even a modest tagset makes this enormous for ordinary sentences. Efficient decoding must reuse partial results instead of exploring every complete path.

34. Greedy decisions can fail

Greedy tagging picks the best current tag from limited context; a locally strong choice can block a better full path. In “One bit me”, “bit” must be a verb in context, even if a local choice favors a noun analysis. Global sequence constraints resolve it.

35. Viterbi search lattice

The lattice shows candidate tags at each word and the transitions between them. A path through “Janet will back the bill” is scored as a whole; Viterbi keeps the best partial path to each state instead of enumerating every full path.

36. Initialize the first token

Viterbi initializes scores for the first word by multiplying start-state transition probabilities by emission probabilities. For “I”, the PRON path receives the strongest nonzero score; impossible word-tag pairs remain at zero.

37. Recur across the next token

For “want”, compute one score per possible current tag: take the best previous-state score, multiply by the transition into the candidate tag and the word’s emission. Keep the maximizing predecessor so the eventual sequence can be reconstructed.

38. Store the best partial path

The table illustrates the recurrence for the second token. Each candidate tag combines a prior path score with the transition and emission for “want”; the winning score is stored for later positions, not just the locally best tag.

39. Continue through the remaining tokens

The next table columns apply the same recurrence to “to” and “race”. For each possible tag, retain only its best incoming path and include the current word’s emission. This compact table carries the evidence from all earlier words forward.

40. Recover the best path

Backtrace from the highest-scoring final state to recover the chosen tags: PRON, VERB, PART, VERB. Backpointers make the globally best sequence recoverable from the compact dynamic-programming table.

41. Practical HMM limits

A bigram HMM only sees the previous tag and struggles to include arbitrary clues such as capitalization (“us” versus “US”). Higher-order tag contexts expand the state space; adding richer features cleanly is a general weakness of generative models.

42. Generative and discriminative models

Generative HMMs model how tags and words are produced. Discriminative models directly model P(T|W) and can use features of the observed sentence. A conditional random field is a discriminative model designed for structured label sequences.

43. CRF roadmap

This section introduces CRFs as a way to keep sequence-level decoding while using more flexible evidence from the input. The lecture then extends the same tagging task from hand-designed features to neural features.

44. CRF conditional model

A linear-chain CRF assigns a globally normalized conditional probability to a tag sequence given the whole word sequence. It sums weighted feature scores along the path, then normalizes against the scores of all candidate tag sequences.

45. CRFs with HMM-like features

CRF features can first mirror a bigram HMM: at each position, score the previous and current tags together with the current word. The chain still enforces dependencies between adjacent tags, but feature weights are learned discriminatively.

46. CRFs with richer features

CRFs can use features of the full input W and position i, not just the current token. Capitalization, suffixes, surrounding words, and other evidence can influence a tag while the model still scores a coherent sequence.

47. Viterbi decoding for a CRF

CRFs can still use Viterbi because the sequence score factors along neighboring tags. The dynamic-programming recurrence uses learned feature-based scores instead of HMM transition and emission probabilities, while keeping the same search strategy.

48. Neural tagging roadmap

Neural tagging replaces hand-crafted feature templates with representations learned from data. A network can encode each token in context, and a structured decoder can use those representations to produce a consistent tag sequence.

49. Learned contextual features

A neural network computes features f_k(W,i) for the word at position i in its sentence. These contextual features can represent evidence that is difficult to specify manually, then feed a tagger or CRF.

50. LSTM sequence context

An LSTM processes a sequence while carrying a hidden state forward, allowing later token representations to depend on earlier words. Its gates regulate what to retain or update, helping it capture longer context than a fixed window.

51. Bidirectional LSTM context

A bidirectional LSTM runs one LSTM left-to-right and another right-to-left. Combining their hidden states gives each token information from preceding and following words, useful when its tag depends on context on either side.

52. BiLSTM-CRF architecture

The BiLSTM produces contextual features for each word; a CRF scores the complete tag sequence and Viterbi decodes the best path. The diagram assigns one label per token, including punctuation, while the CRF coordinates neighboring labels.

53. What each BiLSTM-CRF part does

This design separates representation and structured prediction: the BiLSTM supplies features for the sentence and current word, the CRF computes P(T|W), and Viterbi returns the highest-scoring tag sequence.

54. Transformer-based tagging

Transformer outputs can provide contextual features for each token. One option feeds them to a CRF and uses Viterbi; another applies a softmax independently at each position and chooses the argmax tag, trading sequence-level constraints for simplicity.

55. Transformer sequence labeling

The transformer encoder contextualizes each token, then a tagging head predicts labels such as B-PER, I-PER, O, and B-ORG. The example uses tokenwise argmax over softmax scores; special tokens such as [CLS] are not part of the output entity sequence.

56. Pretraining and fine-tuning

Pretraining learns general language representations from large text collections. Fine-tuning then adapts the pretrained model to tagged examples, so POS or NER can start from useful language patterns instead of learning everything from scratch.

57. Evaluate tagging

Per-word accuracy measures the share of tokens assigned the correct tag. Reported performance varies with language and dataset size; sufficient data can approach human performance. Cross-lingual transfer may help low-resource languages, but cannot remove every difficulty.

58. Summary roadmap

POS tagging is sequence labeling; HMMs provide a generative model, Viterbi decodes the best path, CRFs provide discriminative sequence scores, and neural systems supply richer representations. The remaining slides compare their trade-offs.

59. Model families at a glance

Three approaches stand out: an HMM with Viterbi, a CRF with learned sequence features, and a neural encoder paired with either a CRF or tokenwise softmax. Each balances context, decoding constraints, and flexibility differently.

60. Local and global context

The field moves from local to global decoding and from local to global features. Transformers make token representations rich enough that some systems return to local argmax decoding; stronger contextual features can reduce the need for structured decoding.

61. Acknowledgments

This slide credits the sources that shaped the lecture. The introduction draws mainly on Jurafsky and Martin, while the HMM and Viterbi sections adapt material by Nathan Schneider and other cited researchers.

62. References

The references identify sources behind the lecture’s methods, including BERT and the textbook treatment of speech and language processing. They are the place to consult for formal definitions, derivations, and implementation details.